

SESSION 3 / 3D技術①

本セッションの位置づけとゴール

- 第3回・第4回で3Dを扱う。第3回はスマホセンサーとAR、第4回は点群・3DGS。第5回以降の高さデータの前段にあたる。
- 目標：スマホのセンサーを Web API から直接触り、地理座標とAR空間の対応関係を把握する。
- センサーの生値が座標や描画にどう変換されるかを、外部SDKに任せきりにせず追えるようにする。

本日の実装はすべてブラウザ内で完結する。ネイティブアプリ（ARKit/ARCore）は使わない。

スマホセンサーの全体像

センサー	取得する物理量	Web API	代表的な用途
GNSS（GPS等）	緯度・経度・高度・速度	Geolocation API	現在地・移動追跡
加速度センサー	3軸の加速度	DeviceMotionEvent	歩数・振動・傾き
ジャイロ스코ープ	3軸の角速度	DeviceMotionEvent (rotationRate)	姿勢変化
地磁気センサー	磁北方向	DeviceOrientationEvent	方位
カメラ	映像	getUserMedia (MediaDevices)	ARの背景映像

なぜブラウザから触るのか — 前提の整理

- **配布の容易さ**：URLひとつで動く。アプリストアの審査・インストールが不要。
- **代償**：センサーへのアクセスに制約がある（精度・権限・OS差異）。ここを理解しておくことが本日の主眼。
- **全APIに共通する前提**：Secure Context（HTTPS もしくは localhost）でしか動かない。
- **権限モデルは分かれている**：位置・カメラ・モーション（方位）はそれぞれ別に許可を取る。

SENSORS / API

Geolocation API の基礎

```
// 位置を継続的に監視する
const watchId = navigator.geolocation.watchPosition(
  (pos) => {
    const { latitude, longitude, accuracy } = pos.coords;
    // accuracy は「半径 accuracy メートルの円内にいる確度」
    console.log(latitude, longitude, accuracy);
  },
  (err) => console.error(err.code, err.message),
  { enableHighAccuracy: true, timeout: 10000, maximumAge: 0 }
);
```

- `getCurrentPosition` (1回) / `watchPosition` (継続) の2系統。
- `coords` に含まれる主な値: `latitude` / `longitude` / `accuracy` / `altitude` / `altitudeAccuracy` / `heading` / `speed`。

Geolocation の制約

- HTTPS 必須 (Secure Context)。ローカル開発では localhost のみ例外。
- **精度は環境依存**：屋内・ビル街で大きく劣化する。 `accuracy` は誤差半径 (メートル) で返るので、値として扱う。
- `heading` / `speed` は**移動中のみ有効**なことが多く、静止時は `null`。方位は Geolocation ではなく DeviceOrientation 側で取る。
- 一度権限を拒否されると、再要求はブラウザ設定からになり難しい。最初の要求タイミングの設計が重要。

DeviceOrientationEvent の座標系

端末の姿勢を3つの角度で表す。

- **alpha**：Z軸まわりの回転。 0-360 （≒ 方位に相当）
- **beta**：X軸まわりの回転。 -180-180 （前後の傾き）
- **gamma**：Y軸まわりの回転。 -90-90 （左右の傾き）

実機で値を出しながら端末を傾けて見せると直感的に伝わる。デモ推奨箇所。

方位の取得は端末ごとに挙動が違う

同じ「北」を出すのに、iOSとAndroidで処理が異なる。

```
function extractHeading(e) {  
  if (typeof e.webkitCompassHeading === "number") {  
    // iOS: 0 = 北、時計回り。真北基準の値がそのまま得られる  
    return e.webkitCompassHeading;  
  }  
  if (e.absolute && e.alpha !== null) {  
    // Android(絶対): alpha は反時計回り。方位へ変換する  
    return (360 - e.alpha) % 360;  
  }  
  return null; // 取得不可  
}
```

- iOS の `alpha` は相対値（リスニング開始時が基準）。真北が欲しければ `webkitCompassHeading` を使う。
- Android の `deviceorientation` は絶対方位を保証しない。

パーミッション (iOS 13+ の追加要件)

```
async function enableOrientation() {  
  // iOS 13+ のみ requestPermission が存在する  
  if (typeof DeviceOrientationEvent.requestPermission === "function") {  
    const res = await DeviceOrientationEvent.requestPermission();  
    if (res !== "granted") return;  
  }  
  // Android はこの分岐を通らず、そのまま登録される  
  window.addEventListener("deviceorientationabsolute", extractHeading);  
  window.addEventListener("deviceorientation", extractHeading);  
}
```

- iOS では `requestPermission()` を必ず「ユーザージェスチャ (ボタンタップ等) の中」で呼ぶ。ページ読み込み時の自動実行は拒否される。
- Android は基本的に許可ダイアログ不要 (ただし HTTPS は必須)。

ECOSYSTEM

3つの層 — WebXR / A-Frame / AR.js

- **WebXR Device API**：ブラウザ標準の低レベルAR/VR API。WebGLで描画。
- **A-Frame**：Three.js の上に載る宣言的HTMLフレームワーク。 `<a-scene>` などのタグで3D空間を書ける。
- **AR.js**：A-Frame等と組み合わせるWebARライブラリ。マーカースペースとロケーションベースの2系統を持つ。

「標準API」「3D記述フレームワーク」「ARの位置合わせ」で役割が分かれている、と整理すると見通しがよい。

WebXRの現状と、今回AR.jsを採用する理由

環境	WebXR immersive-ar
Android Chrome（ARCore対応機）	動作する
iPhone / iPad Safari	immersive-ar は利用できない
visionOS Safari	VRのみ。ARは非対応

- **iPhoneとAndroidの両方で同じ教材を動かしたい** → WebXRだけに依存できない。
- そこで、**getUserMedia（カメラ） + DeviceOrientation（方位）** で成立する **AR.js のロケーションベース** を実装の主軸にする。
- WebXRは「標準として概観する」位置づけにとどめる。

iOSのWebXR対応状況はOSアップデートで変わりうる領域。本日時点の前提として扱う。

AR.jsの2系統

- マーカーベース：ARマーカー（Hiroマーカー等）を基準に3Dオブジェクトを配置。屋内・展示・印刷物との連動に向く。
- ロケーションベース：GPS + 方位で **地理座標そのもの** にオブジェクトを配置。屋外・現地調査に向く。

→ 本日のハンズオン②では **ロケーションベース** を使い、観測ポイントのラベルをカメラ映像に重ねて表示する。

第2回までで扱った「地理座標を扱う」感覚が、そのままAR空間の配置につながる。

CASE STUDY

位置情報ベースARの限界 — 生駒市・鳥居ARの事例

移設前の場所に鳥居をAR表示し、一般ユーザーがスマホで体験できるようにするプロジェクト（講師が生駒市向けに実装）。

- 当初は位置情報ベースで、ジオフェンス的な配置を検討した。
- ブラウザの位置精度は数メートル程度で、cm単位の配置は成立しなかった。マーカベースも検討したうえで、最終的に **AR.js内の座標値ベース** の配置に切り替えた。
- QRコード経由でアクセスさせ、ユーザーの立ち位置・姿勢を限定することで実用的な精度を確保した。

Demo: <https://ikominaprij.xsrv.jp/AR/> / <https://ikoma-demo.netlify.app>

ハンズオン②のラベル表示は数メートル精度で足りる用途。cm単位が必要な場合はマ

HIGH PRECISION

位置合わせをどう高精度化するか — VPS

- GPS + 方位だけでは数メートル程度。cm～dm精度が要る場合は **VPS (Visual Positioning System)** を使う。
- 仕組み：事前に作った3Dマップ（点群 / Street View由来）に、カメラ画像の特徴点を照合し、コンピュータビジョンで端末の位置・姿勢（6DoF）を算出する。**幾何的なマッチング** が本体。
- 代表例：
 - **Google ARCore Geospatial API**：Street View由来の3D点群にニューラルネットで照合。Android中心。
 - **Immersal**：事前マップに対しカメラ画像から緯度・経度・高度を返す。Webブラウザ向けSDKあり。

8th Wall (Niantic) のVPSはcm級だったが、OSS化にVPSは含まれず2027年2月に停

Webで「画像認識 × AR」を成立させるには

- **Immersal の Web VPS** (`vps-for-web`) : 事前マッピング (Mapperアプリ / 360カメラ) → ブラウザでカメラ画像から緯度経度・姿勢を取得。IMUで向きを追いつ、連続localizationでSLAM的に追従。iOS/Android両対応・アプリ不要。
- 前提：対象空間を事前に3Dマップ化しておく必要がある (ゼロショットではない)。
- 生駒事例で足りなかった精度は、この方向 (事前マップ + カメラ照合) で埋められる。ただし運用コストとマッピング作業が乗る。

「どこまで自作するか」の判断：ラベル表示なら位置情報ベースで十分。空間に正確に固定したいならVPS。

マルチモーダルモデルはVPSの代わりになるか

- 画像認識モデル（VLM）が得意なのは **意味理解**：看板のOCR、建物の種類、風景の分類。
- VPSが返す **cm単位の6DoF位置姿勢は幾何計算の産物** であり、意味理解とは別系統。モデルは metric な姿勢を返さない。
- 推論のレイテンシ・コストの面でも、リアルタイムの姿勢追跡には不向き。
- 現実的な役割分担：**幾何的な位置決め = VPS、粗い場所推定・意味づけ（OCR・シーン分類） = VLM。**

「モデルを噛ませれば高精度になる」わけではない、という切り分けを押さえておく。